Overview of Data Structures in C# (.NET)

1. What is a Data Structure?

A **Data Structure** is a way of **organizing and storing data in memory so that it can be accessed and modified efficiently**.

Think of it like organizing books in a library:

- If books are randomly placed → hard to find
- If books are arranged in shelves and categories → easy to locate

Similarly, **data structures organize program data efficiently.**

Example

If we want to store **student marks**, we can store them in:

- Array
- List
- Dictionary
- Stack
- Queue

Each structure has **different advantages** depending on how we use the data.

---

2. Why Data Structures are Important

Data structures help in:

1. **Efficient data storage**
2. **Faster searching**
3. **Better performance**
4. **Organized data management**

Example:

Problem	Data Structure
Store list of students	List
Maintain undo operations	Stack
Process tasks in order	Queue
Fast key-value lookup	Dictionary

---

3. Types of Data Structures in C#

Data structures are broadly classified into two categories.

```
Data Structures
  |
  |---- Primitive
  |---- Non Primitive
```

---

4. Primitive Data Structures

These are **basic built-in types** provided by C#.

Examples:

- int
- float
- double
- char
- bool

Example

```
int age = 25;
double salary = 45000.50;
char grade = 'A';
```

bool isActive = true;

These store **single values**.

---

5. Non-Primitive Data Structures

These structures store **multiple values**.

Two types:

1. **Linear Data Structures**
2. **Non-Linear Data Structures**

---

6. Linear Data Structures

Data is arranged **sequentially (one after another)**.

Examples:

- Array
- List
- Stack
- Queue
- Linked List

6.1 Arrays

Array stores **multiple values of the same type**.

Example

```
int[] marks = { 80, 85, 90, 95 };


Console.WriteLine(marks[0]);
```

Output

80

Characteristics

- Fixed size
- Fast access
- Same datatype

6.2 List (Dynamic Array)

List<T> is more flexible than arrays.

- Size can grow dynamically
- Part of **System.Collections.Generic**

Example

```
using System.Collections.Generic;

List<string> students = new List<string>();

students.Add("Rahul");
students.Add("Anita");
students.Add("Kiran");

foreach(var student in students)
{
    Console.WriteLine(student);
}
```

Output

Rahul

Anita

Kiran

Advantages

- Dynamic size
- Easy insertion and deletion

6.3 Stack (LIFO)

Stack follows **Last In First Out** principle.

Example: **Undo operation**

Last inserted item is removed first

Example

```
Stack<int> numbers = new Stack<int>();


numbers.Push(10);

numbers.Push(20);

numbers.Push(30);


Console.WriteLine(numbers.Pop());
```

Output

30

Operations

Operation	Meaning
Push	Insert element
Pop	Remove element
Peek	View top element

6.4 Queue (FIFO)

Queue follows **First In First Out**.

Example: **Ticket counter**

First person in queue gets ticket first

Example

```
Queue<string> customers = new Queue<string>();


customers.Enqueue("Customer1");

customers.Enqueue("Customer2");

customers.Enqueue("Customer3");


Console.WriteLine(customers.Dequeue());
```

Output

Customer1

Operations

Operation	Meaning
Enqueue	Insert element
Dequeue	Remove element
Peek	View first element

6.5 Linked List

A **LinkedList** stores elements in **nodes connected by references**.

Unlike arrays, memory is **not continuous**.

Example

```
LinkedList<string> cities = new LinkedList<string>();
```

```
cities.AddLast("Hyderabad");
cities.AddLast("Chennai");
cities.AddFirst("Delhi");

foreach(var city in cities)
{
    Console.WriteLine(city);
}
```

Output

Delhi

Hyderabad

Chennai

---

7. Non-Linear Data Structures

These do **not store data sequentially**.

Examples:

- Trees
- Graphs
- Hash Tables

7.1 Dictionary (Hash Table)

Dictionary stores **Key-Value pairs**.

Example:

RollNumber → StudentName

Example

```
Dictionary<int, string> students = new Dictionary<int, string>();
```

```
students.Add(1, "Rahul");
students.Add(2, "Anita");
students.Add(3, "Kiran");

Console.WriteLine(students[2]);
```

Output

Anita

Advantages:

- Very fast lookup
- Key-based retrieval

---

8. Data Structures Available in .NET Collections

C# provides collections in the namespace:

System.Collections

System.Collections.Generic

Common Generic Collections:

Data Structure	**Class**
Dynamic Array	List
Key Value	Dictionary<TKey,TValue>
Stack	Stack
Queue	Queue
Linked List	LinkedList
Sorted Data	SortedList

---

9. Real-World Example (Student Management)

Example using **List**

```
class Student
{
    public int Id;
    public string Name;
}

class Program
{
    static void Main()
    {
        List<Student> students = new List<Student>();

        students.Add(new Student { Id = 1, Name = "Rahul" });
        students.Add(new Student { Id = 2, Name = "Anita" });

        foreach (var s in students)
        {
            Console.WriteLine(s.Id + " " + s.Name);
        }
    }
}
```

Output

```
1 Rahul
2 Anita
```

---

10. When to Use Which Data Structure

Scenario	Best Data Structure
Fixed number of items	Array
Dynamic list	List
Undo operations	Stack
Task scheduling	Queue
Key-value lookup	Dictionary
Frequent insert/delete	LinkedList

---

11. Summary

Data structures help us **store and manage data efficiently**.

Common C# data structures:

- Array
- List
- Stack
- Queue
- LinkedList
- Dictionary

They are available through **.NET Collections Framework**.

✅ The **most important structures to master first** are:

1. Arrays
2. List
3. Dictionary

4. Stack
5. Queue
6. LinkedList